UNIT – V

FILES, EXCEPTIONS, MODULES, PACKAGES

Files and exception: text files, reading and writing files, command line arguments, errors and exceptions, handling exceptions, modules (datetime, time, OS , calendar, math module), Explore packages.

Files, Exceptions, Modules, Packages:

Files and exception:

A file is some information or data which stays in the computer storage devices. Python gives you easy ways to manipulate these files. Generally files divide in two categories, text file and binary file. Text files are simple text where as the binary files contain binary data which is only readable by computer.

- **Text files:** In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character (‘\n’) in python by default.
- **Binary files:** In this type of file, there is no terminator for a line and the data is stored after converting it into machine understandable binary language.

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

Text files:

We can create the text files by using the syntax:

Variable name=open (“file.txt”, file mode)

For ex: f= open ("hello.txt","w+")

- We declared the variable f to open a file named hello.txt. **Open** takes 2 arguments, the file that we want to open and a string that represents the kinds of permission or operation we want to do on the file
- Here we used "w" letter in our argument, which indicates write and the plus sign that means it will create a file if it does not exist in library

- The available option beside "w" are "r" for read and "a" for append and plus sign means if it is not there then create it

File Modes in Python:

Mode	Description
'r'	This is the default mode. It Opens file for reading.
'w'	This Mode Opens file for writing. If file does not exist, it creates a new file. If file exists it truncates the file.
'x'	Creates a new file. If file already exists, the operation fails.
'a'	Open file in append mode. If file does not exist, it creates a new file.
't'	This is the default mode. It opens in text mode.
'b'	This opens in binary mode.
'+'	This will open a file for reading and writing (updating)

Reading and Writing files:

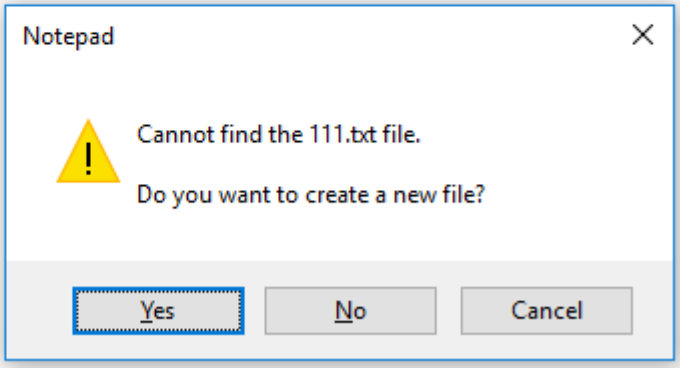
The following image shows how to create and open a text file in notepad from command prompt

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>start notepad hello.txt

C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
Hello mrcet
good morning
how r u
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>
```

(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>notepad 111.txt
```

Hit on enter then it shows the following whether to open or not?



Click on “yes” to open else “no” to cancel

 # Write a python program to open and read a file

a=open(“one.txt”,”r”)

print(a.read())

a.close()

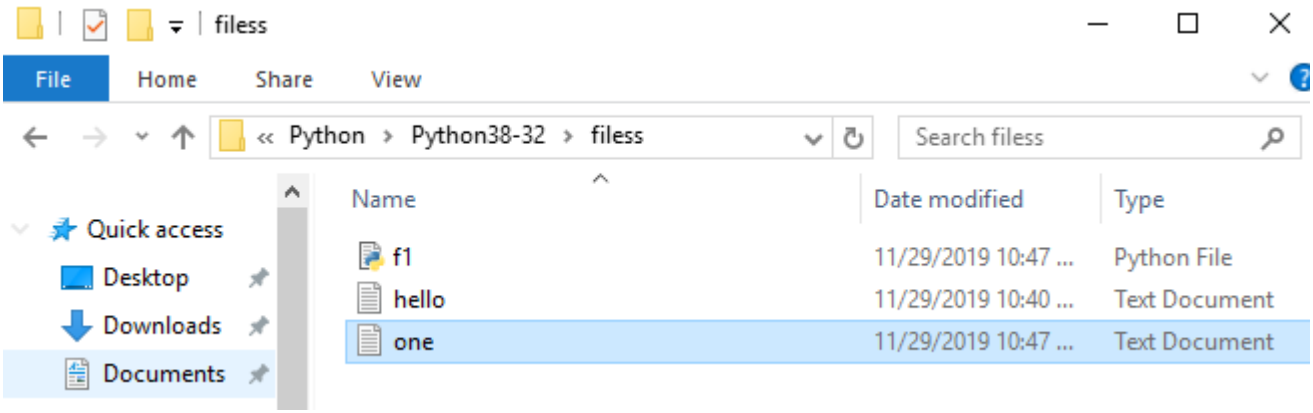
Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/filess/f1.py
welcome to python programming

(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>python f1.py
welcome to python programming
```

Note: All the program files and text files need to saved together in a particular file then only the program performs the operations in the given file mode

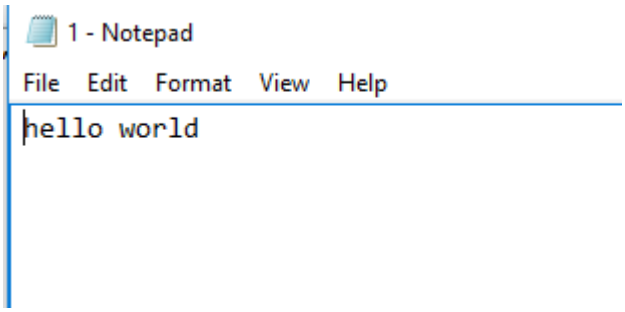


f.close() ---- This will close the instance of the file somefile.txt stored

 # Write a python program to open and write "hello world" into a file?

```
f=open("1.txt","a")
f.write("hello world")
f.close()
```

Output:



(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type 1.txt
hello world
```

Note: In the above program the 1.txt file is created automatically and adds hello world into txt file

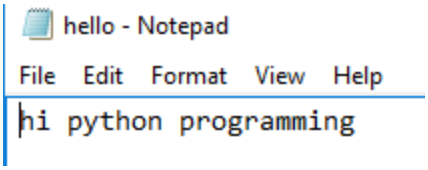
If we keep on executing the same program for more than one time then it append the data that many times

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type 1.txt
hello worldhello world
```

 # Write a python program to write the content "hi python programming" for the existing file.

```
f=open("1.txt",'w')
f.write("hi python programming")
f.close()
```

Output:



In the above program the hello txt file consist of data like

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
Hello mrcet
good morning
how r u
```

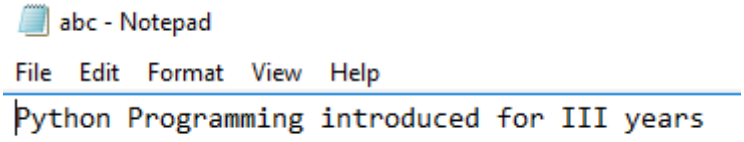
But when we try to write some data on to the same file it overwrites and saves with the current data (check output)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
hi python programming
```

 # Write a python program to open and write the content to file and read it.

```
fo=open("abc.txt","w+")
fo.write("Python Programming")
print(fo.read())
fo.close()
```

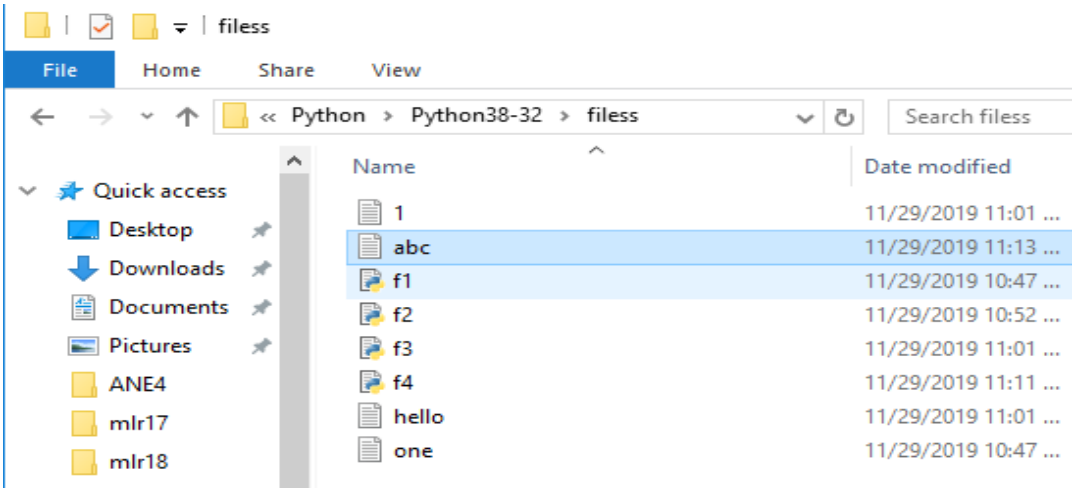
Output:



(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type abc.txt
Python Programming introduced for III years
```

Note: It creates the abc.txt file automatically and writes the data into it



Command line arguments:

The command line arguments must be given whenever we want to give the input before the start of the script, while on the other hand, raw_input() is used to get the input while the python program / script is running.

The command line arguments in python can be processed by using either 'sys' module, 'argparse' module and 'getopt' module.

'sys' module :

Python sys module stores the command line arguments into a list, we can access it using sys.argv. This is very useful and simple way to read command line arguments as String.
sys.argv is the list of commandline arguments passed to the Python program. argv represents all the items that come along via the command line input, it's basically an array holding the command line arguments of our program

>>> sys.modules.keys() -- this prints so many dict elements in the form of list.

```
# Python code to demonstrate the use of 'sys' module for command line arguments

import sys

# command line arguments are stored in the form
# of list in sys.argv
argumentList = sys.argv
print(argumentList)

# Print the name of file
print(sys.argv[0])

# Print the first argument after the name of file
#print(sys.argv[1])
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/cmndlinarg.py

['C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/cmndlinarg.py']
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/cmndlinarg.py
```

Note: Since my list consist of only one element at '0' index so it prints only that list element, if we try to access at index position '1' then it shows the error like,

IndexError: list index out of range

```
import sys

print(type(sys.argv))
print('The command line arguments are:')
for i in sys.argv:
    print(i)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/symod.py ==
<class 'list'>
The command line arguments are:
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/symod.py
```

 # write a python program to get python version.

```
import sys
print("System version is:")
print(sys.version)
print("Version Information is:")
print(sys.version_info)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/s1.py =
System version is:
3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)]
Version Information is:
sys.version_info(major=3, minor=8, micro=0, releaselevel='final', serial=0)
```

'argparse' module :

Python getopt module is very similar in working as the C getopt() function for parsing command-line parameters. Python getopt module is useful in parsing command line arguments where we want user to enter some options too.

```
>>> parser = argparse.ArgumentParser(description='Process some integers.')

#---------
```

```
import argparse

parser = argparse.ArgumentParser()
print(parser.parse_args())
```

'getopt' module :

Python argparse module is the preferred way to parse command line arguments. It provides a lot of option such as positional arguments, default value for arguments, help message, specifying data type of argument etc

It parses the command line options and parameter list. The signature of this function is mentioned below:

getopt.getopt(args, shortopts, longopts=[ ])

- args are the arguments to be passed.
- shortopts is the options this script accepts.
- Optional parameter, longopts is the list of String parameters this function accepts which should be supported. Note that the -- should not be prepended with option names.

-h --------- print help and usage message
-m --------- accept custom option value
-d --------- run the script in debug mode

```
import getopt
import sys

argv = sys.argv[0:]
try:
    opts, args = getopt.getopt(argv, 'hm:d', ['help', 'my_file='])
    #print(opts)
    print(args)
except getopt.GetoptError:
    # Print a message or do something useful
    print('Something went wrong!')
    sys.exit(2)
```